AI Bootcamp — Week 1: Foundations

Monday — Student Exercise Guide

© 2026 Improving

OBJECTIVES

By end of Week 1, the candidate can:

- Explain the Prompt/Harness/Model framework and why the harness, not the model, controls what a skill can do
- Apply the EDD (Evaluation-Driven Development) discipline: write Evaluation Criteria (EC) and manual test cases before building any skill
- Explain and apply the RTCC prompt structure (Role = WHO, Task = WHAT, Context = WHY, Constraints = HOW) to a real skill
- Build four sequential content creation skills as durable, committed `.skill.md` files
- Direct an AI to write a Python evaluation script (deterministic checks plus an LLM judge) and interpret its results
- Verify that every instruction in a skill is load-bearing using manual tests and evaluation runs
- Produce a working end-to-end content pipeline: Trend Identification → Ideation → Development → Graphics

© 2026 Improving

TIMING

- **Monday:** 1:00 PM – 5:00 PM (travel morning, 4 hours)
- **Tuesday–Thursday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each)
- **Friday:** 9:00 AM – 12:00 PM (travel afternoon, 3 hours)

© 2026 Improving

MONDAY (1:00 PM – 5:00 PM): EDD, PROMPT/HARNESS/MODEL, AND RTCC

1:00 – 2:15 PM — EDD: DEFINE SUCCESS BEFORE YOU BUILD

Core concept: The single most important discipline in this program is writing your tests before writing your prompt. Every skill you build this week follows this order:

1. Write the manual test file (Input Prompts + Expected Outputs) by hand. The Expected Outputs are your **Evaluation Criteria (EC)**: the quality bar for the skill's output.
2. Run the test against a baseline prompt. Some tests should fail.
3. Write the skill to make failing tests pass.
4. Verify every instruction is load-bearing; remove any that are not.

The Load-Bearing Rule: If a test passes without an instruction you're about to add, do not add it. Prompts are not documentation. Every word should be doing work that can be proven.

Why manual, not AI-generated test files: When you manually define Expected Outputs, you commit to what success looks like before the model has shaped your expectations. If you let the model generate the test cases, you may end up testing that the model produces what the model would produce anyway, which tells you nothing about whether your instructions are actually altering behavior.

Exercise: Given this bare prompt, `"Identify trending topics in AI on social media"`, write a manual test file with 5 entries. For each entry, specify:

- **Input:** a specific focus/realm the user would provide
- **Expected Output (EC):** describe the *structure and content criteria* the response must meet to pass (not the exact text, the criteria)

Examples of bad EC: *"A list of good topics."* Examples of good EC: *"A numbered list of 5–8 topics. Each entry includes: the topic name, one sentence explaining why it is currently trending, and 3–5 associated hashtags in #format. No topic should be more than 3 months old based on training context. No entry should repeat another."*

Spend 30 minutes writing these 5 test cases. Then run the bare prompt against each. Record which pass and which fail. You should see failures. That is the point.

2:15 – 3:00 PM — PROMPT/HARNESS/MODEL

Before going further, get the conceptual frame that the entire DLP is built on:

- **Prompt:** the instructions you give to a model. What you want it to do, how you want it done, what constraints it must follow.

© 2026 Improving

- **Harness:** the software environment that mediates between you and the model. The harness determines what the model can actually do: read files, run commands, access tools. Claude Code, Devin, GitHub Copilot, and ChatGPT web are all different harnesses with different boundaries.
- **Model:** the AI itself. It generates text based on its training and the prompt, but it cannot do anything on your machine that the harness does not allow.

A model without a harness is a chat interface. The harness is the capability boundary; the model is the intelligence; the prompt is the instruction.

Why this matters this week: the skills you are building carry `disable-model-invocation: true` in their frontmatter. That flag is a harness-level decision. It means the harness will never let the model decide to run your skill on its own; it only runs when you invoke it. You keep invocation control while you build trust in the skill through evaluation evidence. (Week 3 covers the graduation criteria for flipping that flag.)

Discussion: which harness are you running? What can it do that a chat window cannot? What can it not do?

3:00 – 4:15 PM — RTCC: THE PROMPT STRUCTURE

RTCC is the four-part prompt structure used throughout this program:

- **R — Role (WHO):** who the model should act as. A specific, credentialed persona changes tone, vocabulary, and default assumptions.
- **T — Task (WHAT):** the specific action it should perform. Verb-first and unambiguous.
- **C — Context (WHY):** the intent behind the task. Why is this being done, what is it for, what does a good outcome make possible?
- **C — Constraints (HOW):** the data, rules, and specifications the model needs. What the output must contain, what it must never do, what format is required.

Why this split matters: Context is not "background information." It is the *why*. Constraints are not "extra rules." They are the *how*. When an instruction feels out of place during a load-bearing audit, the first diagnostic is structural: is this instruction in the right RTCC section? A rule sitting in Context will be treated as rationale, not enforced as a requirement.

Why this order matters: Role and Task together define the basic contract. Context shapes how the contract is fulfilled. Constraints are guardrails. Adding Constraints before establishing Role and Task often produces over-restricted outputs that satisfy the guardrails but miss the intent.

Exercise: Rewrite the bare trend-identification prompt from earlier as a full RTCC prompt. Draft each element separately before combining:

- R: Who should be doing this identification?
- T: What exactly are they doing?
- C (WHY): Why does this identification matter? What will the output be used for, and by whom?
- C (HOW): What must be true of the output, and what should it never include?

© 2026 Improving

After combining: re-run your 5 manual test cases. How many now pass? For any that still fail, identify which RTCC element is missing or weak.

4:15 – 5:00 PM — REFLECTION

- Review your test file and your RTCC prompt side by side.
- For each instruction in the prompt, remove it mentally: would any test fail? If no test would fail, flag the instruction for removal or replacement.
- For each flagged instruction: is it in the right RTCC section, or is it a Constraint masquerading as Context?
- Discuss: What was hard to define in Expected Output? What made an EC too vague to test against?

© 2026 Improving